

Combining Queries

In this chapter, you'll learn how to use the `UNION` operator to combine multiple `SELECT` statements into one result set.

Understanding Combined Queries

A SQL query usually contains a single `SELECT` statement that returns data from one or more tables. MySQL also enables you to perform multiple queries (multiple `SELECT` statements) and return the results as a single query result set. These combined queries are usually known as *unions* or *compound queries*.

There are basically two scenarios in which you'd use combined queries:

- To return similarly structured data from different tables in a single query
- To perform multiple queries against a single table and return the data as one query

Tip

Combining Queries and Multiple `WHERE` Conditions For the most part, combining two queries to the same table accomplishes the same thing as using a single query with multiple `WHERE` clause conditions. In other words, any `SELECT` statement with multiple `WHERE` clauses can also be specified as a combined query, as you'll see in the section that follows. The performance of each of the two techniques, however, can vary based on the queries used. It is always a good idea to experiment to determine which is preferable in a specific situation.

Creating Combined Queries

You combine SQL queries by using the `UNION` operator. By using `UNION`, you can specify multiple `SELECT` statements, and you can combine their results into a single result set.

Using `UNION`

Using `UNION` is simple enough. All you do is specify each `SELECT` statement and place the keyword `UNION` between them.

Let's look at an example. Say that you need a list of all products costing 5 or less. You also want to include all products made by vendors 1001 and 1002, regardless of price. Of course, you can create a `WHERE` clause to do this, but this time you'll use a `UNION` instead.

As just explained, creating a `UNION` involves writing multiple `SELECT` statements. Here are the individual statements:

► Input

```
SELECT vend_id, prod_id, prod_price
FROM products
WHERE prod_price <= 5;
```

► Output

vend_id	prod_id	prod_price
1003	FC	2.50
1002	FU1	3.42
1003	SLING	4.49
1003	TNT1	2.50

► Input

```
SELECT vend_id, prod_id, prod_price
FROM products
WHERE vend_id IN (1001,1002);
```

► Output

vend_id	prod_id	prod_price
1001	ANV01	5.99
1001	ANV02	9.99
1001	ANV03	14.99
1002	FU1	3.42
1002	OL1	8.99

► Analysis

The first `SELECT` retrieves all products with a price of no more than 5. The second `SELECT` uses `IN` to find all products made by vendors 1001 and 1002.

You can combine these two statements like this:

► Input

```
SELECT vend_id, prod_id, prod_price
FROM products
WHERE prod_price <= 5
```

```
UNION
SELECT vend_id, prod_id, prod_price
FROM products
WHERE vend_id IN (1001,1002);
```

► Output

+-----+-----+-----+			
vend_id	prod_id	prod_price	
+-----+-----+-----+			
1003	FC	2.50	
1002	FU1	3.42	
1003	SLING	4.49	
1003	TNT1	2.50	
1001	ANV01	5.99	
1001	ANV02	9.99	
1001	ANV03	14.99	
1002	OL1	8.99	
+-----+-----+-----+			

► Analysis

This statement is made up of both of the previous `SELECT` statements separated by the `UNION` keyword. `UNION` instructs MySQL to execute both `SELECT` statements and combine the output into a single query result set.

As a point of reference, here is the same query using multiple `WHERE` clauses instead of `UNION`:

► Input

```
SELECT vend_id, prod_id, prod_price
FROM products
WHERE prod_price <= 5
   OR vend_id IN (1001,1002);
```

In this simple example, `UNION` might actually be more complicated than a `WHERE` clause. But with more complex filtering conditions, or if the data is being retrieved from multiple tables (and not just a single table), `UNION` could make the process much simpler.

UNION Rules

As you can see, unions are very easy to use. But a few rules govern exactly which unions can be combined:

- A union must be composed of two or more `SELECT` statements, each separated by the keyword `UNION`. (So, to combine four `SELECT` statements, three `UNION` keywords would be used.)
- Each query in a union must contain the same columns, expressions, or aggregate functions (although columns need not be listed in the same order).
- Column datatypes must be compatible: They need not be exactly the same type, but they must be of a type that MySQL can implicitly convert (for example, different numeric types or different date types).

Aside from these basic rules and restrictions, unions can be used for any data retrieval tasks.

Including or Eliminating Duplicate Rows

Go back to the preceding section, titled “Using UNION,” and look at the sample `SELECT` statements used. Notice that when these statements are executed individually, the first `SELECT` statement returns four rows, and the second `SELECT` statement returns five rows. However, when the two `SELECT` statements are combined with `UNION`, only eight rows are returned—not nine.

`UNION` automatically removes any duplicate rows from the query result set; in other words, it behaves just as multiple `WHERE` clause conditions in a single `SELECT` would. Because vendor 1002 creates a product that costs less than 5, that row is returned by both `SELECT` statements. When `UNION` is used, the duplicate row is eliminated.

This is the default behavior of `UNION`, but you can change it if you so desire. If you do, in fact, want all occurrences of all matches returned, you can use `UNION ALL` instead of `UNION`.

Look at the following example:

► Input

```
SELECT vend_id, prod_id, prod_price
FROM products
WHERE prod_price <= 5
UNION ALL
SELECT vend_id, prod_id, prod_price
FROM products
WHERE vend_id IN (1001,1002);
```

► Output

vend_id	prod_id	prod_price
1003	FC	2.50
1002	FU1	3.42
1003	SLING	4.49
1003	TNT1	2.50
1001	ANV01	5.99
1001	ANV02	9.99
1001	ANV03	14.99
1002	FU1	3.42
1002	OL1	8.99

► Analysis

When you use `UNION ALL`, MySQL does not eliminate duplicates. Therefore, this example returns nine rows, one of them occurring twice.

Tip

UNION Versus WHERE At the beginning of this chapter, I said that **UNION** almost always accomplishes the same thing as multiple **WHERE** conditions. **UNION ALL** is the form of **UNION** that accomplishes what cannot be done with **WHERE** clauses. If you do, in fact, want all occurrences of matches for every condition (including duplicates), you must use **UNION ALL** and not **WHERE**.

Sorting Combined Query Results

SELECT statement output is sorted using the **ORDER BY** clause. When you combine queries with **UNION**, only one **ORDER BY** clause may be used, and it must occur after the final **SELECT** statement. There is very little point in sorting part of a result set one way and part another way, and so multiple **ORDER BY** clauses are not allowed.

The following example sorts the results returned by the previously used union:

► Input

```
SELECT vend_id, prod_id, prod_price
FROM products
WHERE prod_price <= 5
UNION
SELECT vend_id, prod_id, prod_price
FROM products
WHERE vend_id IN (1001,1002)
ORDER BY vend_id, prod_price;
```

► Output

vend_id	prod_id	prod_price
1001	ANV01	5.99
1001	ANV02	9.99
1001	ANV03	14.99
1002	FU1	3.42
1002	OL1	8.99
1003	TNT1	2.50
1003	FC	2.50
1003	SLING	4.49

► Analysis

This union takes a single **ORDER BY** clause after the final **SELECT** statement. Even though the **ORDER BY** appears to only be a part of that last **SELECT** statement, MySQL uses it to sort all the results returned by all the **SELECT** statements.

Note

Combining Different Tables For the sake of simplicity, all of the examples in this chapter combine queries using the same table. However, everything you learned here also applies to using `UNION` to combine queries of different tables.

Summary

In this chapter, you learned how to combine `SELECT` statements with the `UNION` operator. Using `UNION`, you can return the results of multiple queries as one combined query, either including or excluding duplicates. The use of `UNION` can greatly simplify complex `WHERE` clauses and the process of retrieving data from multiple tables.

Challenges

1. Write a SQL statement that combines two `SELECT` statements that retrieve product ID (`prod_id`) and quantity from the `OrderItems` table, one filtering for rows with a quantity of exactly 100 and the other filtering for products with an ID that begins with `BNBG`. Sort the results by product ID.
2. Rewrite the SQL statement you just created to instead use a single `SELECT` statement.
3. This one is a little nonsensical, I know, but it does something you learned in this chapter. Write a SQL statement that returns and combines the product name (`prod_name`) from `Products` and the customer name (`cust_name`) from `Customers` and sort the result by product name.
4. What is wrong with the following SQL statement? (Try to figure it out without running it.)

```
SELECT cust_name, cust_contact, cust_email
FROM Customers
WHERE cust_state = 'MI'
ORDER BY cust_name;
UNION
SELECT cust_name, cust_contact, cust_email
FROM Customers
WHERE cust_state = 'IL'
ORDER BY cust_name;
```